

Fine-tuning a masked language model

অনেক NLP অ্যাপ্লিকেশনে Transformer model ব্যবহার করতে হলে, তুমি Hugging Face Hub থেকে একটা pretrained model নিয়ে তোমার নিজের task-এর data-তে সরাসরি fine-tune করতে পারো।

যদি pretraining-এর জন্য ব্যবহৃত corpus আর fine-tuning-এর corpus খুব বেশি আলাদা না হয়, তাহলে transfer learning সাধারণত ভালো ফল দেয়।

তবে কিছু ক্ষেত্রে task-specific head train করার আগে language model-টাকে আগে তোমার নিজের data-তে fine-tune করা দরকার হয়।

উদাহরণ হিসেবে, যদি তোমার dataset-এ legal contracts বা scientific articles থাকে, তাহলে BERT-এর মতো সাধারণ Transformer model তোমার corpus-এর domain-specific শব্দগুলোকে rare token হিসেবে দেখতে পারে। ফলে performance ততটা ভালো নাও হতে পারে।

এই সমস্যা কমানোর জন্য language model-টাকে আগে তোমার নিজের domain-এর data-তে fine-tune করলে downstream task-গুলোর performance অনেক ভালো হতে পারে। আর ভালো দিক হলো, এই কাজটা সাধারণত একবার করলেই হয়।

Pretrained language model-কে নিজের domain-এর data-তে fine-tune করার এই process-কে সাধারণত **domain adaptation** বলা হয়।

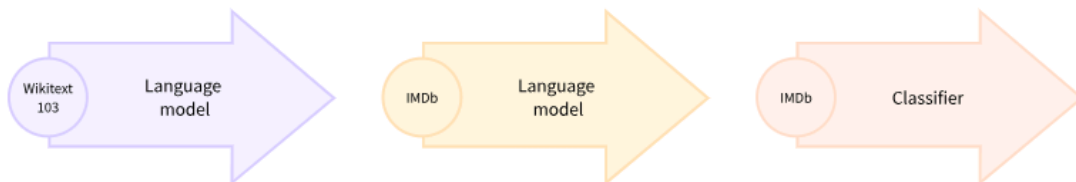
এটা 2018 সালে **ULMFiT**-এর মাধ্যমে জনপ্রিয় হয়। ULMFiT ছিল NLP-তে transfer learning-কে কার্যকরভাবে কাজে লাগানো প্রথম দিকের neural architecture-গুলোর একটি, যা LSTM-এর উপর ভিত্তি করে তৈরি ছিল।

এই section-এ আমরা একই ধরনের কাজ করব, তবে LSTM-এর বদলে Transformer ব্যবহার করে।

এই section শেষ করতে করতে তোমার Hugging Face Hub-এ এমন একটি masked language model থাকবে, যেটা বাক্য autocomplete করতে পারবে।

চলো শুরু করি!

যদি “masked language modeling” আর “pretrained model” শব্দগুলো নতুন লাগে, তাহলে Chapter 1 দেখে আসতে পারো।



Picking a pretrained model for masked language modeling

শুরু করার জন্য masked language modeling-এর উপযোগী একটা pretrained model বেছে নিতে হবে। Hugging Face Hub-এ “Fill-Mask” filter ব্যবহার করলে এমন model-গুলোর তালিকা পাওয়া যায়।

যদিও BERT আর RoBERTa family-এর model সবচেয়ে বেশি ডাউনলোড হয়, আমরা এখানে **DistilBERT** ব্যবহার করব, কারণ এটা অনেক দ্রুত train করা যায় এবং downstream performance-এ খুব কম বা প্রায় কোনো ক্ষতি হয় না।

এই model-টা **knowledge distillation** নামে একটা technique ব্যবহার করে train করা হয়েছে। এখানে বড় একটি “teacher model” যেমন BERT, ছোট একটি “student model”-কে guide করে train করায়। ফলে student model-এর parameter কম হয়, কিন্তু performance অনেকটাই ভালো থাকে।

এখন `AutoModelForMaskedLM` class দিয়ে DistilBERT download করা যাক:

```
from transformers import AutoModelForMaskedLM

model_checkpoint = "distilbert-base-uncased"
model = AutoModelForMaskedLM.from_pretrained(model_checkpoint)
```

এখন `num_parameters()` method দিয়ে দেখা যায় model-টিতে কত parameter আছে:

```
distilbert_num_parameters = model.num_parameters() / 1_000_000
print(f">>> DistilBERT number of parameters: {round(distilbert_num_parameters)}M")
print(f">>> BERT number of parameters: 110M")
```

Output:

```
'>>> DistilBERT number of parameters: 67M'
'>>> BERT number of parameters: 110M'
```

অর্থাৎ DistilBERT-এর প্রায় **67 million** parameter আছে, যেখানে BERT base model-এর আছে **110 million**।

তাই DistilBERT প্রায় ২ গুণ ছোট, আর training-ও প্রায় ২ গুণ দ্রুত হয়।

এখন দেখি, এই model একটা ছোট text-এর `[MASK]` জায়গায় কোন token বসানো সবচেয়ে সম্ভাব্য বলে ধরে:

```
text = "This is a great [MASK]."
```

মানুষ হিসেবে আমরা ভাবতে পারি `[MASK]`-এর জায়গায় “day”, “ride”, “painting” ইত্যাদি আসতে পারে।

কিন্তু pretrained model-এর prediction নির্ভর করে সে কোন corpus-এর উপর train হয়েছে তার উপর। কারণ model data-এর statistical pattern শিখে।

DistilBERT-ও BERT-এর মতো English Wikipedia আর BookCorpus-এর উপর pretrained হয়েছিল, তাই [MASK]-এর prediction-এ ওই domain-এর প্রভাব দেখা যাবে।

Mask predict করতে tokenizer লাগবে, তাই আগে tokenizer-ও download করি:

```
from transformers import AutoTokenizer
```

```
tokenizer = AutoTokenizer.from_pretrained(model_checkpoint)
```

এখন tokenizer আর model ব্যবহার করে text model-এ পার্থানো যাবে, logits বের করা যাবে, আর top 5 candidate দেখা যাবে:

```
import torch
```

```
inputs = tokenizer(text, return_tensors="pt")
token_logits = model(**inputs).logits
# Find the location of [MASK] and extract its logits
mask_token_index = torch.where(inputs["input_ids"] == tokenizer.mask_token_id)[1]
mask_token_logits = token_logits[0, mask_token_index, :]
# Pick the [MASK] candidates with the highest logits
top_5_tokens = torch.topk(mask_token_logits, 5, dim=1).indices[0].tolist()

for token in top_5_tokens:
    print(f">>> {text.replace(tokenizer.mask_token, tokenizer.decode([token]))}")
```

Output:

```
'>>> This is a great deal.'
'>>> This is a great success.'
'>>> This is a great adventure.'
'>>> This is a great idea.'
'>>> This is a great feat.'
```

এই output দেখে বোঝা যায় model সাধারণ everyday English term predict করছে, যা স্বাভাবিক, কারণ এটি English Wikipedia-র মতো সাধারণ domain-এর data-তে train হয়েছে।

এখন দেখা হবে কীভাবে এই domain-কে বদলে আরও niche কিছুতে নেওয়া যায়, যেমন highly polarized movie reviews।

ব্যখ্যা

এখন পুরো জিনিসটা একদম সহজ করে বলি।

ধরো, একটা student আগে থেকেই অনেক বই পড়ে এসেছে। তাই তার সাধারণ ইংরেজি সম্পর্কে ভালো ধারণা আছে। এটাই হলো **pretrained model**।

এখন তুমি যদি ওই student-কে medical, legal, বা scientific লেখা বুঝাতে চাও, তাহলে সে অনেক শব্দ চিনবে না বা ঠিকমতো ধরতে পারবে না।

তাই প্রথমে তাকে ওই নির্দিষ্ট domain-এর বই/ডাটা পড়াও।

এটাই হলো **domain adaptation**।

তারপর যদি তুমি তাকে specific task দাও, যেমন:

- sentiment analysis
- classification
- question answering

তাহলে সে আরও ভালো করবে।

এখানে মূল idea কী?

এই অংশে লেখক বলছে:

সরাসরি **task training** শুরু না করে, আগে **model**-টাকে তোমার নিজের **domain**-এর **text**-এর সাথে মানিয়ে নাও।

কারণ:

- সাধারণ BERT সব ধরনের শব্দ সমানভাবে ভালো জানে না
- বিশেষ domain-এর শব্দকে rare বা unfamiliar মনে করতে পারে
- তাই আগে domain-এর data দিয়ে fine-tune করলে পরে result ভালো আসে

Masked Language Model কী?

এখানে sentence-এর একটা শব্দ লুকানো থাকে, যেমন:

This is a great [MASK].

Model-এর কাজ:

- [MASK]-এর জায়গায় কোন শব্দ সবচেয়ে likely হবে সেটা guess করা

যেমন model বলল:

- deal
- success
- adventure
- idea
- feat

অর্থাৎ model sentence completion করছে।

কেন **DistilBERT** ব্যবহার করা হলো?

কারণ:

- BERT-এর চেয়ে ছোট
- training দ্রুত
- performance প্রায় কাছাকাছি

সংখ্যায়:

- BERT = 110M parameters
- DistilBERT = 67M parameters

মানে কম size, কম computation, বেশি speed।

tokenizer কেন লাগছে?

Model text সরাসরি বোঝে না।

সে number বোঝে।

তাই tokenizer-এর কাজ:

- text কে token-এ ভাঙা
- token কে id-তে convert করা
- model-এর উপযোগী format বানানো

উদাহরণ:

"This is a great [MASK]."

এটা tokenizer ভেঙে model-এর input বানায়।

logits কী?

Model যখন prediction দেয়, তখন সে আগে raw score দেয়।

এই raw score-গুলোই **logits**।

তারপর:

- যেটার score বেশি
- সেটাই বেশি সম্ভাব্য token

এই code-এ top 5 highest score token বের করা হয়েছে।

কোডটা খুব সহজভাবে কী করছে?

1) Model load করছে

```
model = AutoModelForMaskedLM.from_pretrained(model_checkpoint)
```

এখানে masked language modeling-এর জন্য উপযোগী pretrained model আনা হচ্ছে।

2) Tokenizer load করছে

```
tokenizer = AutoTokenizer.from_pretrained(model_checkpoint)
```

Text কে model-এর বোঝার মতো বানানোর tool load হচ্ছে।

3) Text tokenize করছে

```
inputs = tokenizer(text, return_tensors="pt")
```

Sentence কে token id-তে convert করছে।

4) Model prediction নিচ্ছে

```
token_logits = model(**inputs).logits
```

Model সব token-এর জন্য score দিচ্ছে।

5) [MASK] কোথায় আছে সেটা খুঁজছে

```
mask_token_index = torch.where(inputs["input_ids"] == tokenizer.mask_token_id)[1]
```

6) শুধু **[MASK]** পজিশনের **score** নিচ্ছে

```
mask_token_logits = token_logits[0, mask_token_index, :]
```

7) **Top 5 prediction** বের করছে

```
top_5_tokens = torch.topk(mask_token_logits, 5, dim=1).indices[0].tolist()
```

8) **Token** কে **word** বানিয়ে **print** করছে

for token in top_5_tokens:

```
    print(f">>> {text.replace(tokenizer.mask_token, tokenizer.decode([token]))}")
```

Dataset

Domain adaptation দেখানোর জন্য আমরা **Large Movie Review Dataset (IMDb)** ব্যবহার করব। এটা movie review-এর একটি বিখ্যাত corpus, যা sentiment analysis model benchmark করার জন্য অনেক ব্যবহৃত হয়।

DistilBERT-কে এই corpus-এ fine-tune করলে আশা করা যায়, language model তার vocabulary-কে

Wikipedia-র factual data থেকে movie review-এর subjective ভাষার দিকে adapt করবে।
আমরা Hugging Face Hub থেকে `load_dataset()` function দিয়ে data নিতে পারি।

```
from datasets import load_dataset

imdb_dataset = load_dataset("imdb")
imdb_dataset
```

Output:

```
DatasetDict({
  train: Dataset({
    features: ['text', 'label'],
    num_rows: 25000
  })
  test: Dataset({
    features: ['text', 'label'],
    num_rows: 25000
  })
  unsupervised: Dataset({
    features: ['text', 'label'],
    num_rows: 50000
  })
})
```

এখানে দেখা যাচ্ছে:

- `train` split-এ 25,000 review
- `test` split-এ 25,000 review
- `unsupervised` split-এ 50,000 review আছে, যেগুলো unlabeled।

এখন sample দেখে text কেমন তা বোঝা যায়। আগের chapter-গুলোর মতো `Dataset.shuffle()` আর `Dataset.select()` chain করে random sample নেওয়া হচ্ছে।

```
sample = imdb_dataset["train"].shuffle(seed=42).select(range(3))
```

```
for row in sample:
    print(f"\n>>> Review: {row['text']}")
    print(f">>> Label: {row['label']}")
```

এর output-এ কিছু movie review আর তাদের label দেখা যায়। এখান থেকে বোঝা যায়:

- 0 মানে negative review
- 1 মানে positive review

যদিও language modeling-এর জন্য label লাগবে না, তবু শুরুতে data check করে নেওয়া useful।

এখন data-টা masked language modeling-এর জন্য prepare করতে হবে। এটা sequence classification-এর তুলনায় একটু আলাদা।

Preprocessing the data

Auto-regressive আর masked language modeling—দুই ক্ষেত্রেই common preprocessing step হলো:

1. সব example একসাথে concatenate করা
2. তারপর পুরো corpus-কে equal-size chunk-এ ভাগ করা।

এটা আমাদের আগের পদ্ধতি থেকে আলাদা, কারণ আগে আমরা individual example tokenize করতাম। সবকিছু concatenate করার কারণ হলো: কোনো example যদি অনেক বড় হয়, তাহলে আলাদা tokenize করলে truncate হয়ে যেতে পারে, আর তাতে useful information হারিয়ে যেতে পারে।

প্রথমে corpus tokenize করা হবে, তবে `truncation=True` দেওয়া হবে না।

আর tokenizer fast হলে `word_ids`-ও নেওয়া হবে, কারণ পরে whole word masking-এর জন্য এগুলো দরকার হবে।

একই সাথে `text` আর `label` column remove করে দেওয়া হবে।

```
def tokenize_function(examples):
    result = tokenizer(examples["text"])
    if tokenizer.is_fast:
        result["word_ids"] = [result.word_ids(i) for i in range(len(result["input_ids"]))]
    return result
```

```
# Use batched=True to activate fast multithreading!
tokenized_datasets = imdb_dataset.map(
    tokenize_function, batched=True, remove_columns=["text", "label"]
)
tokenized_datasets
```

Output:

```
DatasetDict({
  train: Dataset({
```

```

        features: ['attention_mask', 'input_ids', 'word_ids'],
        num_rows: 25000
    })
    test: Dataset({
        features: ['attention_mask', 'input_ids', 'word_ids'],
        num_rows: 25000
    })
    unsupervised: Dataset({
        features: ['attention_mask', 'input_ids', 'word_ids'],
        num_rows: 50000
    })
})

```

DistilBERT যেহেতু BERT-like model, তাই encoded text-এ আছে:

- `input_ids`
- `attention_mask`
- `word_ids`

এখন tokenized review-গুলোকে group করে chunk-এ ভাগতে হবে। কিন্তু chunk কত বড় হবে?

এটা GPU memory-এর ওপর depend করে। শুরু করার জন্য tokenizer-এর `model_max_length` দেখে maximum context size জানা যায়।

```
tokenizer.model_max_length
```

Output:

512

অর্থাৎ এই model-এর context size 512 token, ঠিক BERT-এর মতো।

কিন্তু Colab-এর মতো GPU-তে fit করানোর জন্য এখানে ছোট chunk নেওয়া হয়েছে:

```
chunk_size = 128
```

তবে বাস্তব জীবনে খুব ছোট chunk size ঋতিকর হতে পারে, তাই use case অনুযায়ী size নিতে হবে।

এখন কয়েকটা tokenized review-এর token length দেখা হচ্ছে:

```
# Slicing produces a list of lists for each feature
```

```
tokenized_samples = tokenized_datasets["train"][:3]
```

```
for idx, sample in enumerate(tokenized_samples["input_ids"]):  
    print(f">>> Review {idx} length: {len(sample)}")
```

Output:

```
'>>> Review 0 length: 200'  
'>>> Review 1 length: 559'  
'>>> Review 2 length: 192'
```

এরপর এগুলো concatenate করা হচ্ছে:

```
concatenated_examples = {  
    k: sum(tokenized_samples[k], []) for k in tokenized_samples.keys()  
}  
total_length = len(concatenated_examples["input_ids"])  
print(f">>> Concatenated reviews length: {total_length}")
```

Output:

```
'>>> Concatenated reviews length: 951'
```

তারপর এই concatenated review-গুলোকে **chunk_size** অনুযায়ী split করা হচ্ছে:

```
chunks = {  
    k: [t[i : i + chunk_size] for i in range(0, total_length, chunk_size)]  
    for k, t in concatenated_examples.items()  
}
```

```
for chunk in chunks["input_ids"]:  
    print(f">>> Chunk length: {len(chunk)}")
```

Output:

```
'>>> Chunk length: 128'  
'>>> Chunk length: 128'  
'>>> Chunk length: 128'  
'>>> Chunk length: 128'  
'>>> Chunk length: 128'  
'>>> Chunk length: 128'
```

```
'>>> Chunk length: 128'
```

```
'>>> Chunk length: 55'
```

শেষ chunk সাধারণত ছোট হয়। এটা handle করার ২টা strategy আছে:

- যদি chunk ছোট হয় তাহলে drop করে দেওয়া
- অথবা pad করে `chunk_size`-এর সমান করা।

এখানে প্রথম strategy নেওয়া হয়েছে। এজন্য সব logic একসাথে `group_texts()` function-এ রাখা হয়েছে:

```
def group_texts(examples):
    # Concatenate all texts
    concatenated_examples = {k: sum(examples[k], []) for k in examples.keys()}
    # Compute length of concatenated texts
    total_length = len(concatenated_examples[list(examples.keys())[0]])
    # We drop the last chunk if it's smaller than chunk_size
    total_length = (total_length // chunk_size) * chunk_size
    # Split by chunks of max_len
    result = {
        k: [t[i : i + chunk_size] for i in range(0, total_length, chunk_size)]
        for k, t in concatenated_examples.items()
    }
    # Create a new labels column
    result["labels"] = result["input_ids"].copy()
    return result
```

এখানে শেষ ধাপে `labels` নামে নতুন column বানানো হয়েছে, যা `input_ids`-এর copy।

কারণ masked language modeling-এ input batch-এর random masked token predict করাই objective, আর `labels` column model-কে ground truth দেয়।

এখন এই function `Dataset.map()` দিয়ে apply করা হচ্ছে:

```
lm_datasets = tokenized_datasets.map(group_texts, batched=True)
lm_datasets
```

Output:

```
DatasetDict({
  train: Dataset({
    features: ['attention_mask', 'input_ids', 'labels', 'word_ids'],
    num_rows: 61289
  })
})
```

```

test: Dataset({
  features: ['attention_mask', 'input_ids', 'labels', 'word_ids'],
  num_rows: 59905
})
unsupervised: Dataset({
  features: ['attention_mask', 'input_ids', 'labels', 'word_ids'],
  num_rows: 122963
})
})

```

এখানে grouping আর chunking করার পর example সংখ্যা অনেক বেড়ে গেছে। কারণ এখন contiguous token নিয়ে নতুন নতুন chunk তৈরি হয়েছে, যেগুলো original corpus-এর একাধিক example জুড়ে হতে পারে।

এটা বোঝার জন্য একটি chunk decode করা হয়েছে:

```
tokenizer.decode(lm_datasets["train"][1]["input_ids"])
```

এখানে [SEP] আর [CLS] token দেখা যাচ্ছে, যা বুঝাচ্ছে একই chunk-এর মধ্যে দুইটা আলাদা review-এর অংশ চলে এসেছে।

এখন labels decode করলে দেখা যায়:

```
tokenizer.decode(lm_datasets["train"][1]["labels"])
```

এটাও input_ids-এর মতোই দেখাচ্ছে।

তাহলে model শিখবে কীভাবে?

কারণ এখনো একটা জরুরি ধাপ বাকি আছে: input-এর random position-এ [MASK] token বসানো। এটা fine-tuning-এর সময় special data collator দিয়ে on-the-fly করা হবে।

ব্যাখ্যা

এখন একদম সহজভাবে বুঝি।

এই অংশের main idea কী?

এখানে লেখক movie review data নিয়ে language model-কে train করার preparation দেখাচ্ছে।

আগে model Wikipedia-র মতো general text দেখে শিখেছিল।
এখন আমরা চাই model movie review-এর ভাষা শিখুক:

- “awesome”
- “boring”
- “waste of time”
- “great acting”
- “terrible plot”

এই জন্য IMDb dataset ব্যবহার করা হচ্ছে।

Dataset-এ কী আছে?

IMDb dataset-এ 3 ভাগ data আছে:

- **train** → training-এর জন্য
- **test** → test করার জন্য
- **unsupervised** → label ছাড়া extra text data

আর label:

- **0 = negative**
- **1 = positive**

কিন্তু এখানে আমাদের main goal sentiment classification না।
আমরা language model fine-tune করছি।
তাই label খুব দরকার নেই, text-টাই আসল।

কেন individual review ধরে tokenize করা হচ্ছে না?

কারণ কিছু review অনেক বড় হতে পারে।

যদি তুমি একেকটা review আলাদা করে tokenize করো, আর max length cross করে যায়, তাহলে বাকিটা কেটে যাবে। এতে important information হারাবে।

তাই কী করা হচ্ছে?

সব review-এর token একসাথে জোড়া লাগানো হচ্ছে, তারপর fixed size chunk-এ কাটা হচ্ছে।

এটা অনেকটা এরকম:

ধরো তোমার কাছে 3টা paragraph আছে।
তুমি আলাদা আলাদা না পড়ে, সব একসাথে জোড়া লাগিয়ে তারপর 128-128 শব্দ করে ভাগ করলা।
এটাই এখানে হচ্ছে, তবে word না, token দিয়ে।

tokenization-এর পর কী পাওয়া যায়?

`tokenize_function()` text-কে model-এর language-এ convert করে।

এখানে পাওয়া যায়:

- `input_ids` → token-এর number
- `attention_mask` → কোন token আসল, কোনটা padding ইত্যাদি
- `word_ids` → কোন token কোন word-এর অংশ, এটা whole-word masking-এ কাজে লাগবে

`chunk_size = 128` মানে কী?

মানে final data-র প্রতিটা training example হবে 128 token-এর।

যেমন:

- প্রথম 128 token = chunk 1
- পরের 128 token = chunk 2
- এভাবে চলবে

শেষে যদি 55 token বাকি থাকে:

- drop করে দিবে, অথবা
- pad করবে

এখানে drop করা হয়েছে।

`group_texts()` function কী করছে?

এই function-এর কাজ 4টা:

1. সব **token** একসাথে জোড়া লাগানো

```
concatenated_examples = {k: sum(examples[k], []) for k in examples.keys()}
```

2. মোট **length** বের করা

```
total_length = len(...)
```

3. **chunk_size** অনুযায়ী কাটা

```
[t[i : i + chunk_size] for i in range(0, total_length, chunk_size)]
```

4. **labels** বানানো

```
result["labels"] = result["input_ids"].copy()
```

labels = input_ids.copy() কেন?

এখানে অনেকেই confuse হয়।

Masked language modeling-এ model-এর কাজ হলো masked word predict করা।

ধরো input ছিল:

This movie is [MASK].

আসল sentence ছিল:

This movie is amazing.

তাহলে:

- input-এ [MASK] থাকবে
- label-এ আসল token থাকবে: amazing

এখনও code-এ mask বসানো হয়নি।

তাই আপাতত **labels** আর **input_ids** same রাখা হয়েছে।

পরে random mask ঢুকানো হবে।

তাহলে এখনো **model** শিখছে না?

ঠিক তাই — এখনো পুরো learning setup complete হয়নি।

কারণ:

- এখন input আর label একই
- কোথাও [MASK] বসানো হয়নি

পরের step-এ data collator random token-গুলো mask করবে।

তখন model শিখবে:

“এই masked জায়গায় আসলে কোন word ছিল?”

Example সংখ্যা এত বেড়ে গেল কেন?

আগে train split-এ ছিল 25,000 review।

কিন্তু পরে হয়ে গেল 61,289 chunk।

কারণ একেকটা বড় review থেকে একাধিক chunk তৈরি হচ্ছে।

আবার একটা chunk দুইটা review-এর অংশ মিলিয়েও হতে পারে।

[SEP] আর [CLS] কেন দেখা যাচ্ছে?

BERT-type model-এ special token ব্যবহার হয়।

- [CLS] → sentence/chunk শুরুতে
- [SEP] → boundary বোঝাতে

যখন decode করে দেখা হলো, তখন বোঝা গেছে এক chunk-এর মধ্যে একাধিক review-এর অংশ এসে গেছে।

Trainer API দিয়ে DistilBERT fine-tune করা

Masked language model fine-tune করা প্রায় sequence classification model fine-tune করার মতোই, যেমন আমরা Chapter 3-এ করেছি।

মূল পার্থক্য শুধু এই যে, এখানে একটা special **data collator** দরকার হয়, যা প্রতিটি batch-এর text-এর কিছু token-কে random ভাবে mask করতে পারে।

ভাগ্য ভালো, Transformers-এ এর জন্য আলাদা **DataCollatorForLanguageModeling** আগে থেকেই আছে।

আমাদের শুধু tokenizer আর **mlm_probability** দিতে হবে, যা বলে দেয় মোট token-এর কত অংশ mask করা হবে।

এখানে আমরা **15%** নেব, কারণ BERT-এও এই পরিমাণ ব্যবহার করা হয়েছিল, আর literature-এ এটিই common choice।

```
from transformers import DataCollatorForLanguageModeling
```

```
data_collator = DataCollatorForLanguageModeling(tokenizer=tokenizer, mlm_probability=0.15)
```

Random masking কীভাবে কাজ করে সেটা দেখতে কয়েকটা example data collator-এ দেওয়া হচ্ছে।

কারণ collator list of dict আশা করে, যেখানে প্রতিটি dict একেকটা contiguous text chunk, তাই আগে dataset থেকে sample নেওয়া হচ্ছে।

এখানে "word_ids" key সরিয়ে ফেলা হচ্ছে, কারণ এই data collator এটা আশা করে না।

```
samples = [lm_datasets["train"][i] for i in range(2)]
```

```
for sample in samples:
```

```
    _ = sample.pop("word_ids")
```

```
for chunk in data_collator(samples)["input_ids"]:
```

```
    print(f'\n>>> {tokenizer.decode(chunk)}')
```

Output-এ দেখা যায় বিভিন্ন জায়গায় random [MASK] token বসে গেছে।

Training-এর সময় model-কে এই masked token-গুলো predict করতে হবে।

আর data collator-এর সুন্দর দিক হলো, প্রতিটি batch-এ mask করার জায়গাগুলো নতুন করে randomize হবে।

একটা side effect হলো, একই data collator train আর test—দুই জায়গায় ব্যবহার করলে evaluation metric deterministic হবে না, কারণ masking প্রতি বার বদলাবে।

পরে 😊 Accelerate দিয়ে custom evaluation loop বানালে randomness freeze করা যায়।

Whole word masking

Masked language modeling-এ আরেকটি technique হলো শুধু token না mask করে, পুরো word একসাথে mask করা।

এটাকে **whole word masking** বলা হয়।

যদি এটা ব্যবহার করতে চাই, তাহলে আমাদের নিজস্ব data collator বানাতে হবে।

Data collator মূলত এমন একটি function, যা sample-এর list নিয়ে batch বানায়।

এখানে আগে compute করা word_ids ব্যবহার করে word index আর তার token-গুলোর মধ্যে mapping বানানো হচ্ছে।

তারপর random ভাবে কিছু word mask করা হচ্ছে।

এক্ষেত্রে labels-এর সব জায়গায় -100 থাকবে, শুধু masked word-এর জায়গাগুলো ছাড়া।

```
import collections
```

```
import numpy as np
```

```

from transformers import default_data_collator

wwm_probability = 0.2

def whole_word_masking_data_collator(features):
    for feature in features:
        word_ids = feature.pop("word_ids")

        # Create a map between words and corresponding token indices
        mapping = collections.defaultdict(list)
        current_word_index = -1
        current_word = None
        for idx, word_id in enumerate(word_ids):
            if word_id is not None:
                if word_id != current_word:
                    current_word = word_id
                    current_word_index += 1
                    mapping[current_word_index].append(idx)

        # Randomly mask words
        mask = np.random.binomial(1, wwm_probability, (len(mapping),))
        input_ids = feature["input_ids"]
        labels = feature["labels"]
        new_labels = [-100] * len(labels)
        for word_id in np.where(mask)[0]:
            word_id = word_id.item()
            for idx in mapping[word_id]:
                new_labels[idx] = labels[idx]
                input_ids[idx] = tokenizer.mask_token_id
        feature["labels"] = new_labels

    return default_data_collator(features)

```

এখন আগের sample-গুলোর ওপর এটি ব্যবহার করা যায়:

```

samples = [lm_datasets["train"][i] for i in range(2)]
batch = whole_word_masking_data_collator(samples)

for chunk in batch["input_ids"]:
    print(f"\n'>>> {tokenizer.decode(chunk)}")

```

Output-এ দেখা যায়, একই word-এর সব token একসাথে masked হচ্ছে।

Training-এর জন্য dataset ছোট করা

এখন যেহেতু fine-tuning-এর বাকি step-গুলো standard, আর Google Colab-এ training সময় নিতে পারে, তাই training set ছোট করা হচ্ছে।

কিছু হাজার example দিয়েও বেশ ভালো language model পাওয়া যাবে।

Datasets-এর `Dataset.train_test_split()` function দিয়ে এটা দ্রুত করা যায়।

```
train_size = 10_000
test_size = int(0.1 * train_size)

downsampled_dataset = lm_datasets["train"].train_test_split(
    train_size=train_size, test_size=test_size, seed=42
)
downsampled_dataset
```

Output:

```
DatasetDict({
  train: Dataset({
    features: ['attention_mask', 'input_ids', 'labels', 'word_ids'],
    num_rows: 10000
  })
  test: Dataset({
    features: ['attention_mask', 'input_ids', 'labels', 'word_ids'],
    num_rows: 1000
  })
})
```

এখানে:

- training set = 10,000 example
 - validation/test set = 1,000 example
- যদি GPU শক্তিশালী হয়, এটা আরও বাড়ানো যায়।

Hugging Face Hub-এ login

এরপর Hugging Face Hub-এ login করতে হবে। Notebook-এ করলে:

```
from huggingface_hub import notebook_login

notebook_login()
```

এতে widget দেখাবে, যেখানে credential দিতে হবে।
অথবা terminal-এ:

```
huggingface-cli login
```

চালানো যায়।

TrainingArguments সেট করা

এখন **Trainer**-এর argument define করা হচ্ছে:

```
from transformers import TrainingArguments

batch_size = 64
# Show the training loss with every epoch
logging_steps = len(downsampled_dataset["train"]) // batch_size
model_name = model_checkpoint.split("/")[-1]

training_args = TrainingArguments(
    output_dir=f"{model_name}-finetuned-imdb",
    overwrite_output_dir=True,
    evaluation_strategy="epoch",
    learning_rate=2e-5,
    weight_decay=0.01,
    per_device_train_batch_size=batch_size,
    per_device_eval_batch_size=batch_size,
    push_to_hub=True,
    fp16=True,
    logging_steps=logging_steps,
)
```

এখানে কিছু important option বদলানো হয়েছে:

- **evaluation_strategy="epoch"** → প্রতি epoch শেষে evaluation
- **learning_rate=2e-5**

- `weight_decay=0.01`
- `batch_size=64`
- `push_to_hub=True` → trained model Hub-এ push হবে
- `fp16=True` → mixed precision training, ফলে speed বাড়ে

আরও একটা গুরুত্বপূর্ণ বিষয়:

`Trainer` by default model-এর `forward()` method-এ না থাকা column remove করে দেয়।

তাই whole word masking collator ব্যবহার করলে `remove_unused_columns=False` দিতে হবে, নাহলে `word_ids` column হারিয়ে যাবে।

এছাড়া `hub_model_id` argument দিয়ে Hub-এ কোন repository-তে push হবে সেটা নিজে specify করা যায়।

না দিলে default হিসেবে নিজের namespace-এ `output_dir`-এর নাম অনুযায়ী repo তৈরি হবে।

Trainer বানানো

এখন সব ingredient প্রস্তুত, তাই `Trainer` instantiate করা হচ্ছে।

এখানে standard `data_collator` ব্যবহার করা হয়েছে, তবে exercise হিসেবে whole word masking collator-ও try করা যেতে পারে।

```
from transformers import Trainer
```

```
trainer = Trainer(
    model=model,
    args=training_args,
    train_dataset=downsampled_dataset["train"],
    eval_dataset=downsampled_dataset["test"],
    data_collator=data_collator,
    tokenizer=tokenizer,
)
```

এখন `trainer.train()` চালানোর জন্য সব প্রস্তুত।

তবে তার আগে language model-এর performance মাপার একটি common metric **perplexity** নিয়ে briefly আলোচনা করা হবে।

ব্যাখ্যা

এখন একদম easy করে বুঝি।

এই অংশের main idea কী?

আগের অংশে আমরা data prepare করেছি।

এই অংশে দেখাচ্ছে:

কীভাবে আসল **training** শুরু হবে।

মানে:

- random mask বসানো
- batch বানানো
- training setup তৈরি করা
- Trainer দিয়ে fine-tune ready করা

Data collator কী?

Data collator হলো training-এর ঠিক আগে batch বানানোর helper।

এটার কাজ:

- sample-গুলো একসাথে batch বানানো
- দরকার হলে padding করা
- এখানে extra কাজ হিসেবে random [MASK] বসানো

DataCollatorForLanguageModeling কী করছে?

এটা automatically কিছু token random ভাবে mask করছে।

```
data_collator = DataCollatorForLanguageModeling(tokenizer=tokenizer, mlm_probability=0.15)
```

এখানে `mlm_probability=0.15` মানে:

- মোট token-এর 15% mask হবে

ধরো 100টা token আছে

তাহলে প্রায় 15টা token [MASK] হবে।

Masking কেন দরকার?

কারণ masked language model-এর কাজই হলো missing word predict করা।

যেমন:

The movie was [MASK].

Model guess করবে:

- amazing
- terrible
- boring
- fantastic

অর্থাৎ [MASK] না থাকলে model-এর কাজই থাকবে না।

random masking প্রতি batch-এ বদলায় কেন?

এটাই এই method-এর powerful দিক।

একই sentence আজকে এক জায়গায় mask হবে, পরের batch-এ অন্য জায়গায় mask হতে পারে।

ফলে model একই sentence থেকে বারবার নতুনভাবে শিখতে পারে।

word_ids কেন pop করা হলো?

কারণ সাধারণ `DataCollatorForLanguageModeling` শুধু standard input চায়।

ওটা `word_ids` নিয়ে কাজ করে না।

তাই:

```
_ = sample.pop("word_ids")
```

দিয়ে `word_ids` সরানো হয়েছে।

Whole word masking আবার কী?

সাধারণ masking-এ একটা word-এর শুধু এক অংশ mask হতে পারে।

ধরো word = **playing**

tokenized হলে হতে পারে:

- **play**
- **##ing**

সাধারণ masking-এ শুধু **play** mask হতে পারে, **##ing** নাও হতে পারে।

কিন্তু **whole word masking**-এ:

- পুরো word-এর সব token একসাথে mask হবে

Whole word masking collator কীভাবে কাজ করছে?

এই custom function-এর কাজ:

1. **word_ids** দিয়ে কোন **token** কোন **word**-এর অংশ তা খুঁজে বের করা
2. **Random** ভাবে কিছু **word** বাছাই করা
3. ওই **word**-এর সব **token**-কে **[MASK]** বানানো
4. **labels**-এ শুধু **masked** জায়গার আসল **token** রাখা
5. অন্য সব জায়গায় **-100** রাখা

-100 কেন ব্যবহার করা হয়?

এটা খুব important।

PyTorch loss function-এ **-100** মানে:

এই **position ignore** করো

অর্থাৎ model loss calculate করবে শুধু masked জায়গাগুলোতে।
অন্য জায়গাগুলোতে loss ধরবে না।

Training set ছোট করা হলো কেন?

কারণ full dataset-এ training অনেক slow হতে পারে, বিশেষ করে Colab-এ।

তাই:

- 10,000 training example
- 1,000 validation example

নেওয়া হয়েছে।

এতে training দ্রুত হবে, তবুও decent model পাওয়া যাবে।

Hugging Face login কেন লাগছে?

কারণ trained model Hub-এ upload করা হবে।

যেহেতু `push_to_hub=True`, তাই login ছাড়া push করা যাবে না।

TrainingArguments-এ important জিনিসগুলো কী?

`output_dir`

trained model কোথায় save হবে

`evaluation_strategy="epoch"`

প্রতি epoch শেষে evaluation হবে

`learning_rate=2e-5`

model কত দ্রুত শিখবে

`weight_decay=0.01`

overfitting কমাতে সাহায্য করে

per_device_train_batch_size=64

একবারে GPU-তে কত sample যাবে

fp16=True

half precision ব্যবহার করে training দ্রুত করে

push_to_hub=True

training শেষে model Hugging Face Hub-এ upload হবে

remove_unused_columns=False কেন লাগতে পারে?

Whole word masking collator **word_ids** ব্যবহার করে।

কিন্তু Trainer default ভাবে unused column মুছে দেয়।

তাই যদি custom whole word masking ব্যবহার করো, তাহলে **word_ids** যেন delete না হয়, সেজন্য:

remove_unused_columns=False

দিতে হবে।

Trainer object কী?

Trainer হলো Hugging Face-এর ready-made training manager।

এটা:

- training loop চালায়
- evaluation করে
- logging করে
- checkpoint save করে
- Hub-এ push করতে পারে

এই code-এ:

```
trainer = Trainer(  
    model=model,  
    args=training_args,  
    train_dataset=downsampled_dataset["train"],  
    eval_dataset=downsampled_dataset["test"],  
    data_collator=data_collator,  
    tokenizer=tokenizer,  
)
```

মানে Trainer-কে বলা হচ্ছে:

- কোন model train করবে
- কোন dataset use করবে
- কোন settings মানবে
- কীভাবে masking করবে

কোড **flow** একদম সহজভাবে

পুরো flow টা হলো:

Step 1:

data collator বানাও
→ random mask বসাবে

Step 2:

চাইলে whole word masking-এর custom collator বানাও

Step 3:

training dataset ছোট করো
→ training fast হবে

Step 4:

Hugging Face Hub-এ login করো

Step 5:

TrainingArguments set করো

Step 6:

Trainer বানাও

Step 7:

`trainer.train()` চালাও

Language model-এর জন্য Perplexity

Text classification বা question answering-এর মতো task-এ আমাদের কাছে labeled corpus থাকে। কিন্তু language modeling-এ এমন explicit label থাকে না।

তাহলে কীভাবে বুঝব একটা language model ভালো কি না?

মোবাইলের autocorrect feature-এর মতো চিন্তা করো:

একটা ভালো language model সেইটা, যেটা ব্যাকরণগতভাবে সঠিক sentence-কে high probability দেয়, আর অদ্ভুত বা nonsense sentence-কে low probability দেয়।

অনলাইনে অনেক “autocorrect fail” উদাহরণ আছে, যেখানে phone model মজার বা অনুপযুক্ত completion দিয়েছে।

যদি আমাদের test set-এর sentence-গুলো বেশিরভাগই grammatically correct হয়, তাহলে language model-এর quality মাপার একটা উপায় হলো test set-এর sentence-গুলোর next word-এর জন্য model যে probability দেয় তা হিসাব করা।

যদি probability বেশি হয়, তার মানে model unseen example দেখে খুব “surprised” বা “perplexed” হচ্ছে না।

এতে বোঝা যায় model language-এর grammar-এর basic pattern শিখেছে।

Perplexity-এর বিভিন্ন mathematical definition আছে, তবে এখানে আমরা যেটা ব্যবহার করছি সেটা হলো **cross-entropy loss**-এর **exponential**।

তাই pretrained model-এর perplexity বের করতে `Trainer.evaluate()` দিয়ে test set-এ cross-entropy loss compute করা হবে, তারপর তার exponential নেওয়া হবে।

```
import math
```

```
eval_results = trainer.evaluate()  
print(f">>> Perplexity: {math.exp(eval_results['eval_loss']):.2f}")
```

Output:

Perplexity: 21.75

কম perplexity মানে ভালো language model।

এখানে শুরুতে score কিছুটা বড় দেখা যাচ্ছে।

এখন দেখি fine-tuning করে এটা কমানো যায় কি না। প্রথমে training loop চালানো হচ্ছে:

```
trainer.train()
```

তারপর আগের মতোই আবার perplexity compute করা হচ্ছে:

```
eval_results = trainer.evaluate()  
print(f">>> Perplexity: {math.exp(eval_results['eval_loss']):.2f}")
```

Output:

Perplexity: 11.32

এটা বেশ ভালো কমে গেছে, যা দেখাচ্ছে model movie review domain সম্পর্কে কিছু শিখেছে।

Training শেষ হলে model card-সহ training information Hub-এ push করা যায়:

```
trainer.push_to_hub()
```

Accelerate দিয়ে DistilBERT fine-tune করা

Trainer দিয়ে যেমন দেখলাম, masked language model fine-tune করা Chapter 3-এর text classification example-এর মতোই।

মূল সূক্ষ্ম পার্থক্য শুধু special data collator ব্যবহার করা, যা আগেই আলোচনা করা হয়েছে।

তবে আগেই দেখা গেছে `DataCollatorForLanguageModeling` evaluation-এর সময়ও random masking করে।

ফলে প্রতি training run-এ perplexity score কিছুটা ওঠানামা করতে পারে।

এই randomness কমানোর একটি উপায় হলো test set-এ একবার masking apply করে ফেলা, তারপর evaluation-এর সময় default data collator ব্যবহার করা।

এটা দেখানোর জন্য একটি function বানানো হয়েছে, যা batch-এর ওপর masking apply করে:

```
def insert_random_mask(batch):
    features = [dict(zip(batch, t)) for t in zip(*batch.values())]
    masked_inputs = data_collator(features)
    # Create a new "masked" column for each column in the dataset
    return {"masked_" + k: v.numpy() for k, v in masked_inputs.items()}
```

এরপর test set-এ function apply করা হচ্ছে এবং unmasked column-গুলো drop করে masked column-গুলো rename করা হচ্ছে:

```
downsampled_dataset = downsampled_dataset.remove_columns(["word_ids"])
eval_dataset = downsampled_dataset["test"].map(
    insert_random_mask,
    batched=True,
    remove_columns=downsampled_dataset["test"].column_names,
)
eval_dataset = eval_dataset.rename_columns(
    {
        "masked_input_ids": "input_ids",
        "masked_attention_mask": "attention_mask",
        "masked_labels": "labels",
    }
)
```

এখন dataloader usual way-তে setup করা যায়, তবে evaluation set-এর জন্য `default_data_collator` ব্যবহার হবে:

```
from torch.utils.data import DataLoader
from transformers import default_data_collator

batch_size = 64
train_dataloader = DataLoader(
```

```

    downsampled_dataset["train"],
    shuffle=True,
    batch_size=batch_size,
    collate_fn=data_collator,
)
eval_dataloader = DataLoader(
    eval_dataset, batch_size=batch_size, collate_fn=default_data_collator
)

```

এরপর Accelerate-এর standard step follow করা হচ্ছে।
প্রথমে pretrained model-এর fresh version load করা হচ্ছে:

```
model = AutoModelForMaskedLM.from_pretrained(model_checkpoint)
```

তারপর optimizer define করা হচ্ছে, এখানে standard AdamW ব্যবহার করা হয়েছে:

```

from torch.optim import AdamW

optimizer = AdamW(model.parameters(), lr=5e-5)

```

এরপর **Accelerator** object দিয়ে training-এর জন্য সব প্রস্তুত করা হচ্ছে:

```

from accelerate import Accelerator

accelerator = Accelerator()
model, optimizer, train_dataloader, eval_dataloader = accelerator.prepare(
    model, optimizer, train_dataloader, eval_dataloader
)

```

এখন learning rate scheduler define করা হচ্ছে:

```

from transformers import get_scheduler

num_train_epochs = 3
num_update_steps_per_epoch = len(train_dataloader)
num_training_steps = num_train_epochs * num_update_steps_per_epoch

lr_scheduler = get_scheduler(
    "linear",
    optimizer=optimizer,
    num_warmup_steps=0,

```



```
    num_training_steps=num_training_steps,  
)
```

Training শুরু করার আগে Hugging Face Hub-এ model repository তৈরি করতে হবে।
প্রথমে repo-এর full name তৈরি করা হচ্ছে:

```
from huggingface_hub import get_full_repo_name  
  
model_name = "distilbert-base-uncased-finetuned-imdb-accelerate"  
repo_name = get_full_repo_name(model_name)  
repo_name
```

Output:

```
'lewtun/distilbert-base-uncased-finetuned-imdb-accelerate'
```

তারপর **Repository** class দিয়ে repo create ও clone করা হচ্ছে:

```
from huggingface_hub import Repository  
  
output_dir = model_name  
repo = Repository(output_dir, clone_from=repo_name)
```

এরপর training এবং evaluation loop পুরোপুরি নিজে লেখা হচ্ছে:

```
from tqdm.auto import tqdm  
import torch  
import math  
  
progress_bar = tqdm(range(num_training_steps))  
  
for epoch in range(num_train_epochs):  
    # Training  
    model.train()  
    for batch in train_dataloader:  
        outputs = model(**batch)  
        loss = outputs.loss  
        accelerator.backward(loss)  
  
        optimizer.step()  
        lr_scheduler.step()
```

```

optimizer.zero_grad()
progress_bar.update(1)

# Evaluation
model.eval()
losses = []
for step, batch in enumerate(eval_dataloader):
    with torch.no_grad():
        outputs = model(**batch)

    loss = outputs.loss
    losses.append(accelerator.gather(loss.repeat(batch_size)))

losses = torch.cat(losses)
losses = losses[: len(eval_dataset)]
try:
    perplexity = math.exp(torch.mean(losses))
except OverflowError:
    perplexity = float("inf")

print(f">>> Epoch {epoch}: Perplexity: {perplexity}")

# Save and upload
accelerator.wait_for_everyone()
unwrapped_model = accelerator.unwrap_model(model)
unwrapped_model.save_pretrained(output_dir, save_function=accelerator.save)
if accelerator.is_main_process:
    tokenizer.save_pretrained(output_dir)
    repo.push_to_hub(
        commit_message=f"Training in progress epoch {epoch}", blocking=False
    )

```

Output:

```

Epoch 0: Perplexity: 11.397545307900472
Epoch 1: Perplexity: 10.904909330983092
Epoch 2: Perplexity: 10.729503505340409

```

এখানে দেখা যাচ্ছে প্রতি epoch-এর সাথে perplexity কমেছে, এবং multiple training run reproducible হচ্ছে।

Fine-tuned model ব্যবহার করা

Fine-tuned model-কে Hub-এর widget দিয়ে বা locally **pipeline** ব্যবহার করে ব্যবহার করা যায়। এখানে **fill-mask** pipeline দিয়ে model load করা হচ্ছে:

```
from transformers import pipeline

mask_filler = pipeline(
    "fill-mask", model="huggingface-course/distilbert-base-uncased-finetuned-imdb"
)
```

তারপর আগের sample text **"This is a great [MASK]"** model-এ দেওয়া হচ্ছে:

```
preds = mask_filler(text)

for pred in preds:
    print(f">>> {pred['sequence']}")
```

Output:

```
'>>> this is a great movie.'
'>>> this is a great film.'
'>>> this is a great story.'
'>>> this is a great movies.'
'>>> this is a great character.'
```

এখান থেকে বোঝা যায় model এখন movie-related word predict করছে, অর্থাৎ এটি domain-এর সাথে adapt করেছে।

এইভাবেই language model training-এর প্রথম experiment শেষ হলো।

Section 6-এ scratch থেকে GPT-2-এর মতো auto-regressive model train করার কথা শেখানো হবে।

ব্যাখ্যা

এখন একদম easy করে বুঝি।

এই অংশের **main idea** কী?

এই অংশে ৩টা বড় জিনিস আছে:

- model ভালো হলো কিনা সেটা **perplexity** দিয়ে মাপা
- **Trainer** দিয়ে training চালানো
- চাইলে **Accelerate** দিয়ে আরও control নিয়ে custom training loop লেখা
- তারপর fine-tuned model use করে prediction দেখা

Perplexity কী?

Perplexity হলো language model-এর confusion score।

সহজভাবে:

- কম **perplexity = model** কম **confused = ভালো model**
- বেশি **perplexity = model** বেশি **confused = খারাপ model**

ধরো model sentence দেখে মনে করছে:

“আরে, পরের word কী হবে বুঝতে পারছি না!”

তাহলে perplexity বেশি হবে।

আর যদি model confidently next word ধরতে পারে, তাহলে perplexity কম হবে।

কেন **perplexity** দরকার?

Language modeling-এ তো class label নেই।

যেমন sentiment analysis-এ থাকে:

- positive
- negative

কিন্তু এখানে model-এর কাজ next token বা masked token predict করা।

তাই accuracy দিয়ে সবসময় সহজে বিচার করা যায় না।

সেই জন্য perplexity ব্যবহার হয়।

`math.exp(eval_loss)` কেন?

কারণ এখানে perplexity define করা হয়েছে:

perplexity = exp(cross-entropy loss)

তাই:

- আগে `trainer.evaluate()` দিয়ে `eval_loss` নেওয়া
- তারপর `math.exp()` দিয়ে exponent নেওয়া হচ্ছে

শুরুতে **21.75**, পরে **11.32** — এর মানে কী?

এটা খুব important।

Training-এর আগে:

- perplexity = **21.75**

Training-এর পরে:

- perplexity = **11.32**

মানে model এখন আগের চেয়ে অনেক কম confused।

অর্থাৎ:

- movie review domain শিখেছে
- “movie”, “film”, “story”, এসব শব্দ/context এখন ভালো ধরতে পারছে

`trainer.train()` আসলে কী করছে?

এটা actual training loop চালায়।

এখানে model:

- batch নেয়
- masked word predict করে
- loss calculate করে

- weight update করে
- ধীরে ধীরে better হয়

trainer.push_to_hub() কী করছে?

এটা trained model Hugging Face Hub-এ upload করে।

ফলে:

- পরে আবার load করা যাবে
- অন্যরা use করতে পারবে
- model card-ও save হবে

তাহলে আবার Accelerate কেন?

কারণ Trainer সহজ, কিন্তু limited control দেয়।

Accelerate ব্যবহার করলে:

- training loop নিজে লিখতে পারো
- evaluation কাস্টমাইজ করতে পারো
- randomness control করতে পারো
- multi-GPU / mixed precision সহজে চালাতে পারো

Random masking evaluation-এ problem কেন?

আগের data collator evaluation-এর সময়ও random mask বসায়।

ফলে:

- আজকে test set-এর এক জায়গা mask হলো
- কালকে অন্য জায়গা mask হলো

তাই perplexity score একটু বদলে যেতে পারে।

মানে result পুরো deterministic না।

সমাধান কী?

Test set-এ একবারই masking apply করে দাও।

তারপর evaluation-এর সময় fixed masked version ব্যবহার করো।

এতে:

- একই evaluation set থাকবে
- ফল stable হবে
- perplexity run-to-run consistent হবে

`insert_random_mask()` কী করছে?

এই function:

- batch নেয়
- data collator দিয়ে mask বসায়
- masked version return করে

মানে evaluation-এর জন্য আগে থেকেই masked test set বানিয়ে রাখছে।

dataloader দুইটা আলাদা কেন?

`train_dataloader`

এখানে random masking চলবে

তাই `collate_fn=data_collator`

`eval_dataloader`

এখানে fixed masked dataset use হবে

তাই `collate_fn=default_data_collator`

এতে training random, evaluation stable।

Accelerator কী করছে?

```
accelerator = Accelerator()  
model, optimizer, train_dataloader, eval_dataloader = accelerator.prepare(...)
```

এটা training setup-কে device-ready বানায়।

মানে:

- CPU/GPU handle করে
- multi-GPU support করে
- mixed precision support দেয়
- distributed setup সহজ করে

AdamW কী?

এটা optimizer।

Optimizer-এর কাজ:

- loss দেখে
- model-এর weight update করা
- যাতে model ভালো শিখে

AdamW Transformer training-এ খুব common choice।

Learning rate scheduler কী করছে?

Training-এর সময় learning rate ধীরে ধীরে কমাতে বা control করতে scheduler ব্যবহার হয়।

এখানে:

```
get_scheduler("linear", ...)
```

মানে learning rate linearভাবে adjust হবে।

training loop টা একদম সহজ করে

প্রতি **epoch**-এ ২টা **phase**

1) Training

- `model.train()`
- batch নেয়
- loss বের করে
- backpropagation করে
- optimizer step নেয়
- learning rate update হয়

2) Evaluation

- `model.eval()`
- gradient বন্ধ
- loss collect করে
- mean loss থেকে perplexity বের করে

তারপর:

- model save
- Hub-এ push

`accelerator.backward(loss)` কেন?

এটা normal `loss.backward()`-এর accelerate-friendly version।

distributed বা mixed precision training-এ এটা safer/useful।

`accelerator.gather(...)` কেন?

যদি multiple device থাকে, সব device-এর loss একসাথে gather করতে হয়।

তাহে final perplexity সঠিকভাবে বের হয়।

Output-এ epoch অনুযায়ী perplexity কমছে মানে কী?

Epoch 0: 11.39

Epoch 1: 10.90

Epoch 2: 10.73

এটা healthy sign।

মানে:

- model প্রতিটা epoch-এ improve করছে
- movie review domain আরও ভালো শিখছে

Fine-tuned model use করলে কী দেখা গেল?

আগে generic model “great” এর পরে সাধারণ শব্দ দিত:

- success
- idea
- feat

কিন্তু fine-tuned model দিল:

- movie
- film
- story
- character

মানে model এখন movie review domain-এর style ধরেছে।
এটাই domain adaptation-এর success।

একদম সহজ analogy

ধরো একজন student আগে general English শিখেছে।

তারপর তুমি তাকে অনেক movie review পড়ালে।
এখন তুমি বললো:

“This is a great ____”

সে এখন “movie” বা “film” বলবে।
আগে হয়তো “idea” বা “success” বলত।

এটাই fine-tuning-এর effect।